

Principles of Programming Languages

Logic Programming. Rewriting.

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Paradigms

Outline

Paradigms

Logic Programming

Outline

Paradigms

Logic Programming

Rewriting

Outline

Paradigms

Logic Programming

Rewriting

Quick retrospective

Paradigms

- ▶ Imperative Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Declarative Programming
 - ▶ Functional Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Declarative Programming
 - ▶ Functional Programming: ✓
 - ▶ Logic Programming

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Declarative Programming
 - ▶ Functional Programming: ✓
 - ▶ Logic Programming
 - ▶ Rewriting

General facts about PLs

General facts about PLs

We have seen PLs where the computations are based on:

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ higher-order + recursion

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ higher-order + recursion

Are there other approaches?

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ higher-order + recursion

Are there other approaches?

- ▶ Yes

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ higher-order + recursion

Are there other approaches?

- ▶ Yes: Logic programming

Some background

Some background

- ▶ Computation vs. Deduction

Some background

- ▶ Computation vs. Deduction
 - ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)

Some background

- ▶ Computation vs. Deduction
 - ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
 - ▶ Deduction: requires more than automatic application of rules

Some background

- ▶ Computation vs. Deduction
 - ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
 - ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete

Some background

- ▶ Computation vs. Deduction
 - ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
 - ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
 - ▶ Example: $a^2 + b^2 = c^2$

Some background

- ▶ Computation vs. Deduction
 - ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
 - ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
 - ▶ Example: $a^2 + b^2 = c^2$... Pythagoras $\rightarrow$ first proof

Some background: FOL

Some background: FOL

Reminder:

Some background: FOL

Reminder:

- ▶ Terms:

- ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables
- ▶ Predicates: $p(t_1, \dots, t_n)$ – where p is a predicate of arity n

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables
- ▶ Predicates: $p(t_1, \dots, t_n)$ – where p is a predicate of arity n
- ▶ Formulas: $\phi := p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \wedge \phi \mid \exists x.\phi$

Some background

Some background

Reminder:

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{ RULENAME}$$

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{ RULENAME}$$

- Examples:

$$\frac{}{\text{nat}(0)} \text{ ZERO}$$

$$\frac{\text{nat}(N)}{\text{nat}(s(N))} \text{ SUCC}$$

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{ RULENAME}$$

- Examples:

$$\frac{}{\text{nat}(0)} \text{ ZERO}$$

$$\frac{\text{nat}(N)}{\text{nat}(s(N))} \text{ SUCC}$$

$$\frac{}{\text{even}(0)} \text{ EVEN0}$$

$$\frac{\text{even}(N)}{\text{even}(s(s(N)))} \text{ EVENSUCC}$$

Proofs

- ▶ Proof that $\text{even}(s(s(s(s(0)))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{\text{even}(0)} \text{EVEN0}}{\text{even}(s(s(0)))} \text{EVENSUCC}}{\text{even}(s(s(s(s(0))))} \text{EVENSUCC}}$$

Proofs

- ▶ Proof that $even(s(s(s(s(0))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{even(0)} \text{ EVEN0}}{even(s(s(0)))} \text{ EVENSUCC}}{even(s(s(s(s(0))))} \text{ EVENSUCC}}$$

- ▶ Proof strategies: *goal-directed* vs. *forward-reasoning*

Proofs

- ▶ Proof that $even(s(s(s(s(0))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{even(0)} \text{ EVEN0}}{even(s(s(0)))} \text{ EVENSUCC}}{even(s(s(s(0))))} \text{ EVENSUCC}}$$

- ▶ Proof strategies: *goal-directed* vs. *forward-reasoning*
- ▶ **Logic programming** uses *goal-directed*: for the current goal, search the rule that might have been applied to arrive at this conclusion

$$\frac{\cdot}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Logic programming: general facts

- ▶ This paradigm is based on formal logic

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*
- ▶ Logic programming can be seen as “controlled” deduction

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*
- ▶ Logic programming can be seen as “controlled” deduction
- ▶ Logic programming languages (or families of PL): Prolog, ASP (Answer set programming), Datalog

Logic programming: how it works

- ▶ Prolog: early 1970, Alain Colmerauer and Robert Kowalski
- ▶ Demo in Prolog: goals.

$$\frac{}{nat(0)} \text{ ZERO} \qquad \frac{nat(N)}{nat(s(N))} \text{ SUCC}$$

Logic programming: how it works

- ▶ Prolog: early 1970, Alain Colmerauer and Robert Kowalski
- ▶ Demo in Prolog: goals.

$$\frac{}{nat(0)} \text{ ZERO} \qquad \frac{nat(N)}{nat(s(N))} \text{ SUCC}$$

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

- ▶ Proof search: goal-directed

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution
- ▶ Let $T_1 = g(h(z, y), x, y)$ be a term

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution
- ▶ Let $T_1 = g(h(z, y), x, y)$ be a term
- ▶ Then, $\sigma(T_1) = \sigma(g(h(z, y), x, y)) = g(h(f(a, b), y'), 3, y')$.

Unification

- ▶ When proving, the conclusion of an inference rule *matches/unifies with* the current goal.

Unification

- ▶ When proving, the conclusion of an inference rule *matches/unifies with* the current goal.
- ▶ How?
- ▶ Unification algorithm
 - ▶ A substitution σ is the *unifier* of t_1 and t_2 if $\sigma(t_1) = \sigma(t_2)$.
- ▶ Various algorithms for unification (Montanari, Robinson, ...)

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0}$$

$$\frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0}$$

$$\frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

- ▶ Backtracking: more than one possible rule applicable

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

- ▶ Backtracking: more than one possible rule applicable
- ▶ Subgoal order: which of the premises to consider?

Demo

Light meal problem

► Facts:

```
meat (steak, 10) .  
meat (pork, 12) .  
fish (tuna, 2) .  
fish (trout, 4) .  
starter (salad, 1) .  
starter (soup, 3) .  
desert (fruit, 1) .  
desert (cake, 10) .  
desert (icecream, 4) .
```

Demo

Light meal problem

- ▶ Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```

Demo

Light meal problem

- ▶ Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```

- ▶ A simple meal:

```
meal(S, M, D) :- starter(S, SK),  
                  main_course(M, MK),  
                  desert(D, DK).
```

Demo

Light meal problem

- ▶ Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```

- ▶ A simple meal:

```
meal(S, M, D) :- starter(S, SK),  
                  main_course(M, MK),  
                  desert(D, DK).
```

- ▶ A light meal:

```
light_meal(S, M, D) :- starter(S, SK),  
                        main_course(M, MK),  
                        desert(D, DK),  
                        SK + MK + DK < 10.
```

Resources

Reading material on logic programming:

- ▶ **Chapter/Lecture 1:** <https://people.mpi-sws.org/~dg/teaching/lis2014/modules/lp-fp-07.pdf>

Rewriting

Key concepts:

- ▶ sorts
- ▶ equations
- ▶ rewriting rules

A rewrite engine: Maude

- ▶ Maude modules = rewrite theories which consist of: a syntax for terms, equations, and rewrite rules
- ▶ Reduction equations are assumed to be confluent and terminating
- ▶ Rewriting rules: $l \Rightarrow r \text{ if } c$, where l and r are terms with variables, and c is a condition
- ▶ A rewrite rule $l \Rightarrow r \text{ if } c$ can be applied to a term t if l is *matched* by t and the condition is evaluated to true
 - ▶ the result of the matching operation is a substitution σ , s.t., $\sigma(l) = t$
 - ▶ if $\sigma(c)$ is true, then the result of rewriting t with $l \Rightarrow r \text{ if } c$ is $t' = \sigma(r)$
 - ▶ therefore, a rule $l \Rightarrow r \text{ if } c$ yields a transition $t \rightarrow_{l \Rightarrow r \text{ if } c} t'$

A vending machine (*All About Maude* book)

```
fmod VENDING-MACHINE-SYNTAX is
  sorts Coin Item Marking .

  subsorts Coin Item < Marking .
  op _ : Marking Marking ->
      Marking [assoc comm id: null] .
  op null : -> Marking .

  op $ : -> Coin [format (r! o)] .
  op q : -> Coin [format (r! o)] .
  op a : -> Coin [format (b! o)] .
  op c : -> Coin [format (b! o)] .
endfm
```

A vending machine (*All About Maude* book)

```
mod VENDING-MACHINE is
  protecting VENDING-MACHINE-SYNTAX .

  var M : Marking .
  rl [add-q] : M => M q .
  rl [add-$] : M => M $ .
  rl [buy-a] : $ M => a q M .
  rl [buy-c] : $ M => c M .
  rl [change]: q q q q => $ .
endm
```

Several simple rewrites

```
rewrite [2] in VENDING-MACHINE : $ $ q q .
rewrites: 2 in 0ms cpu (0ms real) (68965 rewrites/sec)
result Marking: $ $ $ q q q
=====
rewrite [3] in VENDING-MACHINE : $ $ q q .
rewrites: 3 in 0ms cpu (0ms real) (375000 rewrites/sec)
result Marking: $ $ q q q q a
=====
rewrite [4] in VENDING-MACHINE : $ $ q q .
rewrites: 4 in 0ms cpu (0ms real) (4000000 rewrites/sec)
result Marking: $ q q q q a c
=====
rewrite [5] in VENDING-MACHINE : $ $ q q .
rewrites: 5 in 0ms cpu (0ms real) (714285 rewrites/sec)
result Marking: $ $ a c
```

Search without add-q and add-\$

```
=====
search in VENDING-MACHINE : $ q q q =>! a c M .
```

```
Solution 1 (state 5)
```

```
states: 6 rewrites: 13 in 0ms cpu (0ms real) (171052
rewrites/second)
```

```
M -> null
```

```
No more solutions.
```

```
states: 6 rewrites: 13 in 0ms cpu (0ms real) (139784
rewrites/second)
```

Search: path exploration

```
Maude> show path 5 .  
state 0, Marking: $ q q q  
===[ rl $ M => a q M [label buy-a] . ]===>  
state 1, Marking: q q q q a  
===[ rl q q q q => $ [label change] . ]===>  
state 3, Marking: $ a  
===[ rl $ M => c M [label buy-c] . ]===>  
state 5, Marking: a c
```

Conditional search

=====

```
search in VENDING-MACHINE : $ q q q =>! a M such that  
M != null = true .
```

Solution 1 (state 4)

```
states: 6 rewrites: 14 in 0ms cpu (0ms real) (162790  
rewrites/second)
```

```
M -> q a
```

Solution 2 (state 5)

```
states: 6 rewrites: 15 in 0ms cpu (0ms real) (135135  
rewrites/second)
```

```
M -> c
```

No more solutions.

```
states: 6 rewrites: 15 in 0ms cpu (0ms real) (119047  
rewrites/second)
```

Resources

Reading material on Maude/rewriting logic:

- ▶ *All About Maude:*

`https://maude.cs.illinois.edu/w/images/0/0d/Maude-book.pdf`

Retrospective

- ▶ Basics: Algebraic Data Types; Recursive Functions, Induction, proofs
- ▶ Higher-Order Functions, Logic
- ▶ Syntax: BNF, derivations, ambiguities, ASTs
- ▶ Interpreters
- ▶ Big-step and Small-step SOS
- ▶ Types systems
- ▶ Compilation
- ▶ Paradigms: OOP, Functional Programming, Logic Programming, Rewriting